

Lab: In-Memory Data Structures as Databases

Database Design — Python Review (No Web / No Flask)

Goal: Practice lists, dictionaries, and tuples by modeling and querying a tiny “database” in memory. You’ll see why databases exist: memory is temporary, files are clunky, and structure matters.

What you’ll practice

- Representing tabular data (rows/columns) using lists, dictionaries, and tuples
- Filtering (“WHERE”), projecting columns (“SELECT”), updating rows, and inserting rows
- Building a simple index (dictionary keyed by id) for fast lookups
- Reasoning about in-memory vs on-disk data

Rules / Constraints

- Use only core Python (no pandas, no Flask, no web).
- Write small helper functions when you repeat yourself.
- Add short comment answers for reflection prompts.
- If you finish early, do the Challenge Extension at the end.

Part 1 (): Lists as Tables

Idea: Each row is a list. Columns are positions (0, 1, 2...). This is fast, but the schema is implicit and fragile.

Starter data

```
students = [  
    [1, "Alice", "GSAS", 32],  
    [2, "Bob", "Tandon", 28],  
    [3, "Carol", "GSAS", 40],  
    [4, "Dan", "CAS", 18]  
]  
# columns: [id, name, school, credits]
```

Tasks

- 1 Print all student names (one per line).
- 2 Print only the students in GSAS (id and name).
- 3 Print the students with credits > 30 (name and credits).
- 4 Insert a new student row for: id=5, name='Eve', school='CAS', credits=22.
- 5 Update Bob's credits to 30.
- 6 Reflection (comment): What breaks if someone inserts a new column in the middle?

Part 2 (): Dictionaries as Rows

Idea: Each row is a dictionary. Keys are column names. This makes the schema explicit and the code easier to read.

Starter data

```
students = [  
    {"id": 1, "name": "Alice", "school": "GSAS", "credits": 32},  
    {"id": 2, "name": "Bob", "school": "Tandon", "credits": 28},  
    {"id": 3, "name": "Carol", "school": "GSAS", "credits": 40},  
    {"id": 4, "name": "Dan", "school": "CAS", "credits": 18}  
]
```

Tasks

- 1 Print all student names.
- 2 Print only GSAS students (name + credits).
- 3 Add a new field to each row: status='full-time' if credits >= 30 else 'part-time'.
- 4 Update Bob's credits to 30 and ensure his status updates correctly.
- 5 Write a function: get_students_by_school(rows, school) → list of rows.
- 6 Reflection (comment): Why is dict-based code safer than index-based list code?

Part 3 (): Tuples as Records

Idea: A tuple is immutable. In databases, “tuple” can also mean a row/record. Here we use tuples to see what immutability feels like.

Starter data

```
students = [  
    (1, "Alice", "GSAS", 32),  
    (2, "Bob", "Tandon", 28),  
    (3, "Carol", "GSAS", 40)  
]  
# columns: (id, name, school, credits)
```

Tasks

- 1 Print the name for each record.
- 2 Try to change Bob's credits directly (and observe the error).
- 3 Create a new tuple for Bob with credits=30 and replace the old record in the list.
- 4 Reflection (comment): Why might immutability be a good thing for data integrity?

Part 4 (): Indexing (Fast Lookup by id)

Idea: A dictionary can act like an index: instead of searching all rows to find id=3, we jump directly to it.

Starter code

```
students = [
    {"id": 1, "name": "Alice", "school": "GSAS", "credits": 32},
    {"id": 2, "name": "Bob", "school": "Tandon", "credits": 30},
    {"id": 3, "name": "Carol", "school": "GSAS", "credits": 40},
    {"id": 4, "name": "Dan", "school": "CAS", "credits": 18}
]

# Build an index: id -> student_row
index_by_id = {}
for row in students:
    index_by_id[row["id"]] = row
```

Tasks

- 1 Use index_by_id to print the record for id=3.
- 2 Write a function: find_by_id(index, id) → row or None.
- 3 Insert a new student into both students and the index.
- 4 Update a student's credits using the index (and show that students list reflects the same row).
- 5 Reflection (comment): Which is faster conceptually: scanning a list vs dictionary lookup? Why?

Wrap■Up (): In Memory vs On Disk

- What happens to your in-memory “database” when the program ends?
- Why might a CSV/TSV feel like a “database” but still be annoying?
- What features does a real DB add (persistence, constraints, concurrency, indexing, querying language)?

Challenge Extension (fast finishers)

- **1) Build a school index:** school → list of students. Then print all GSAS students without scanning the full list each time.
- **2) Enforce a constraint:** prevent duplicate ids. If an insert uses an existing id, reject it with a clear message.
- **3) Mini “query” function:** query(rows, school=None, min_credits=None) → filtered rows. Support both filters together.
- **4) Persist to disk (optional):** write the updated rows back to a TSV file using the csv module (tab delimiter).